

JavaScript Object Practice Questions

40+ Complete Collection for Beginners

Introduction

This comprehensive guide contains 40+ JavaScript object questions and practical problems designed to build your expertise from basic concepts to advanced implementations. Each question includes detailed explanations and hands-on coding exercises.

Part 1: Fundamental Concepts (Questions 1-15)

1. What is an object in JavaScript?

Answer: An object is a collection of key-value pairs where keys are strings (or Symbols) and values can be any data type including functions, arrays, and other objects.

Practice Problem: Create a book object with properties: `title`, `author`, `pages`, and `published`.

```
const book = {  
  title: "The Great Gatsby",  
  author: "F. Scott Fitzgerald",  
  pages: 180,  
  published: 1925  
};
```

2. How do you create an empty object?

Answer:

- Object literal: `const obj = {};`
- Constructor: `const obj = new Object();`

Practice Problem: Create an empty `student` object, then add properties `name`, `age`, and `grade` one by one.

3. How do you access object properties?

Answer:

- Dot notation: `obj.property`
- Bracket notation: `obj["property"]`

Practice Problem: Access all properties from this object and print them:

```
const car = { make: "Toyota", model: "Camry", year: 2020 };
```

4. How do you add new properties to an object?

Answer:

- `obj.newProperty = value`
- `obj["newProperty"] = value`

Practice Problem: Add `color` and `mileage` properties to the car object above.

5. How do you delete properties from an object?

Answer: `delete obj.property`

Practice Problem: Remove the `year` property from the car object and display the updated object.

6. How do you check if a property exists?

Answer:

- `"property" in obj`
- `obj.hasOwnProperty("property")`

Practice Problem: Write a function that checks if a `person` object has both `email` and `phone` properties.

7. How do you get all keys from an object?

Answer: `Object.keys(obj)`

Practice Problem: Get all property names from this object:

```
const phone = {  
  brand: "Apple",  
  model: "iPhone",  
  storage: "128GB",  
  color: "black"  
};
```

8. How do you get all values from an object?

Answer: `Object.values(obj)`

Practice Problem: Get all values from the phone object above and find the total character count.

9. How do you loop through object properties?

Answer:

```
for (let key in obj) {  
  console.log(key + ": " + obj[key]);  
}
```

Practice Problem: Loop through a `grades` object and calculate the average score:

```
const grades = {  
  math: 85,  
  english: 92,  
  science: 78,  
  history: 88  
};
```

10. How do you clone an object?

Answer: `const copy = {...obj}` (shallow copy)

Practice Problem: Clone a `user` object and modify the copy without affecting the original.

11. How do you merge two objects?

Answer: `const merged = {...obj1, ...obj2}`

Practice Problem: Merge these two objects:

```
const basic = { name: "John", age: 30 };  
const details = { city: "NYC", job: "Developer" };
```

12. How do you use variables as property names?

Answer: `obj[variableName] = value`

Practice Problem: Create an object using variables for property names:

```
let prop1 = "firstName";  
let prop2 = "lastName";  
// Create object with these as keys
```

13. What is object destructuring?

Answer: `const {name, age} = person;`

Practice Problem: Destructure name and email from this object:

```
const user = {
  name: "Alice",
  email: "alice@test.com",
  age: 25,
  city: "Boston"
};
```

14. How do you check if an object is empty?

Answer: `Object.keys(obj).length === 0`

Practice Problem: Write a function that returns `true` if an object has no properties.

15. How do you convert object to JSON and back?

Answer:

- To JSON: `JSON.stringify(obj)`
- From JSON: `JSON.parse(jsonString)`

Practice Problem: Convert this object to JSON string and then back to object:

```
const data = {
  users: ["John", "Jane"],
  active: true,
  count: 2
};
```

Part 2: Practical Applications (Questions 16-40+)

16. Student Grade Manager

Create a comprehensive student object with grade management capabilities:

```
const student = {
  name: "John Doe",
  grades: [],

  addGrade(subject, score) {
    this.grades.push({ subject, score });
  },

  getAverage() {
```

```

        const total = this.grades.reduce((sum, grade) => sum + grade.score, 0);
        return total / this.grades.length;
    },

    getLetterGrade() {
        const avg = this.getAverage();
        if (avg >= 90) return 'A';
        if (avg >= 80) return 'B';
        if (avg >= 70) return 'C';
        return 'D';
    }
};

```

17. Inventory Management System

Build a complete inventory tracking system:

```

const inventory = {
    items: {},

    addItem(name, quantity, price) {
        this.items[name] = { quantity, price };
    },

    removeItem(name) {
        delete this.items[name];
    },

    updateQuantity(name, quantity) {
        if (this.items[name]) {
            this.items[name].quantity = quantity;
        }
    },

    getTotalValue() {
        let total = 0;
        for (let item in this.items) {
            total += this.items[item].quantity * this.items[item].price;
        }
        return total;
    }
};

```

18. Bank Account Object

Implement a banking system with transaction tracking:

```

const bankAccount = {
    accountNumber: "123456789",
    balance: 0,
    transactions: [],

    deposit(amount) {

```

```

        this.balance += amount;
        this.transactions.push({
            type: 'deposit',
            amount,
            date: new Date()
        });
    },

    withdraw(amount) {
        if (amount <= this.balance) {
            this.balance -= amount;
            this.transactions.push({
                type: 'withdraw',
                amount,
                date: new Date()
            });
        } else {
            console.log("Insufficient funds");
        }
    },

    getStatement() {
        return this.transactions;
    }
};

```

19. Contact Book System

Create a comprehensive contact management system:

```

const contactBook = {
    contacts: {},

    addContact(name, phone, email) {
        this.contacts[name] = { phone, email };
    },

    findContact(name) {
        return this.contacts[name] || "Contact not found";
    },

    updateContact(name, updates) {
        if (this.contacts[name]) {
            Object.assign(this.contacts[name], updates);
        }
    },

    listAllContacts() {
        return Object.keys(this.contacts);
    }
};

```

20. Shopping Cart with Discounts

Build an e-commerce shopping cart system:

```
const cart = {
  items: [],
  discountRate: 0,

  addItem(product, price, quantity) {
    this.items.push({ product, price, quantity });
  },

  removeItem(product) {
    this.items = this.items.filter(item => item.product !== product);
  },

  applyDiscount(rate) {
    this.discountRate = rate;
  },

  calculateTotal() {
    const subtotal = this.items.reduce((sum, item) =>
      sum + (item.price * item.quantity), 0);
    return subtotal * (1 - this.discountRate);
  }
};
```

21. Employee Management System

Develop an HR management object:

```
const employee = {
  id: 1,
  name: "John Smith",
  department: "IT",
  salary: 50000,

  promote(newSalary, newDepartment) {
    this.salary = newSalary;
    if (newDepartment) this.department = newDepartment;
  },

  getAnnualSalary() {
    return this.salary * 12;
  },

  getInfo() {
    return `${this.name} works in ${this.department} and earns $$${this.salary}`;
  }
};
```

22. Weather Station Data

Create a weather monitoring system:

```
const weatherStation = {
  readings: [],

  addReading(temperature, humidity, date) {
    this.readings.push({
      temperature,
      humidity,
      date: date || new Date()
    });
  },

  getAverageTemp() {
    const total = this.readings.reduce((sum, reading) =>
      sum + reading.temperature, 0);
    return total / this.readings.length;
  },

  getHighestTemp() {
    return Math.max(...this.readings.map(r => r.temperature));
  },

  getReadingsAbove(temp) {
    return this.readings.filter(reading => reading.temperature > temp);
  }
};
```

23. Task Management System

Build a comprehensive task tracker:

```
const taskManager = {
  tasks: [],

  addTask(title, priority = 'medium', dueDate) {
    this.tasks.push({
      id: Date.now(),
      title,
      priority,
      dueDate,
      completed: false
    });
  },

  completeTask(id) {
    const task = this.tasks.find(t => t.id === id);
    if (task) task.completed = true;
  },

  getPendingTasks() {
    return this.tasks.filter(task => !task.completed);
  }
};
```



```

    },

    getTasksByPriority(priority) {
        return this.tasks.filter(task => task.priority === priority);
    }
};

```

24. Library Management System

Implement a complete library system:

```

const library = {
    books: [],

    addBook(title, author, isbn) {
        this.books.push({ title, author, isbn, available: true });
    },

    borrowBook(isbn) {
        const book = this.books.find(b => b.isbn === isbn);
        if (book && book.available) {
            book.available = false;
            return `You borrowed "${book.title}"`;
        }
        return "Book not available";
    },

    returnBook(isbn) {
        const book = this.books.find(b => b.isbn === isbn);
        if (book) {
            book.available = true;
            return `"${book.title}" returned successfully`;
        }
    },

    searchByAuthor(author) {
        return this.books.filter(book =>
            book.author.toLowerCase().includes(author.toLowerCase()));
    }
};

```

25. Recipe Management System

Create a cooking recipe manager:

```

const recipe = {
    name: "Chocolate Cake",
    ingredients: [],
    instructions: [],
    servings: 8,

    addIngredient(name, amount, unit) {
        this.ingredients.push({ name, amount, unit });
    }
};

```

```

    },

    addInstruction(step) {
      this.instructions.push(step);
    },

    scaleRecipe(newServings) {
      const ratio = newServings / this.servings;
      this.ingredients.forEach(ingredient => {
        ingredient.amount *= ratio;
      });
      this.servings = newServings;
    }
  };

```

Advanced Utility Functions

26. Object Property Counter

```

function countObjectProperties(obj, includeNested = false) {
  let count = 0;
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      count++;
      if (includeNested && typeof obj[key] === 'object' && obj[key]
        count += countObjectProperties(obj[key], true);
      }
    }
  }
  return count;
}

```

27. Object Validator

```

function validateObject(obj, schema) {
  const errors = [];
  for (let field in schema) {
    const rule = schema[field];
    if (rule.required && !(field in obj)) {
      errors.push(`${field} is required`);
    }
    if (field in obj && rule.type && typeof obj[field] !== rule.type)
      errors.push(`${field} must be of type ${rule.type}`);
    }
  }
  return { isValid: errors.length === 0, errors };
}

```

28. Deep Object Cloning

```
function deepClone(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (obj instanceof Date) return new Date(obj.getTime());
  if (obj instanceof Array) return obj.map(item => deepClone(item));

  const cloned = {};
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      cloned[key] = deepClone(obj[key]);
    }
  }
  return cloned;
}
```

29. Object Comparison

```
function deepEqual(obj1, obj2) {
  if (obj1 === obj2) return true;
  if (obj1 == null || obj2 == null) return false;
  if (typeof obj1 !== 'object' || typeof obj2 !== 'object') return false;

  const keys1 = Object.keys(obj1);
  const keys2 = Object.keys(obj2);

  if (keys1.length !== keys2.length) return false;

  for (let key of keys1) {
    if (!keys2.includes(key)) return false;
    if (!deepEqual(obj1[key], obj2[key])) return false;
  }
  return true;
}
```

30. Object to Array Converter

```
function objectToArray(obj, format = 'entries') {
  switch (format) {
    case 'keys': return Object.keys(obj);
    case 'values': return Object.values(obj);
    case 'entries': return Object.entries(obj);
    case 'pairs': return Object.entries(obj).map(([k, v]) =>
      ({ key: k, value: v }));
    default: return Object.entries(obj);
  }
}
```

Advanced Design Patterns

31. Configuration Manager

```
const config = {
  settings: {},

  set(key, value) {
    this.settings[key] = value;
  },

  get(key, defaultValue = null) {
    return this.settings[key] !== undefined ?
      this.settings[key] : defaultValue;
  },

  has(key) {
    return key in this.settings;
  },

  remove(key) {
    delete this.settings[key];
  },

  getAll() {
    return { ...this.settings };
  }
};
```

32. Cache Implementation

```
const cache = {
  data: {},
  maxSize: 100,

  set(key, value, ttl = 0) {
    if (Object.keys(this.data).length >= this.maxSize) {
      const oldestKey = Object.keys(this.data)[0];
      delete this.data[oldestKey];
    }

    this.data[key] = {
      value,
      timestamp: Date.now(),
      ttl: ttl > 0 ? Date.now() + ttl : 0
    };
  },

  get(key) {
    const item = this.data[key];
    if (!item) return null;

    if (item.ttl > 0 && Date.now() > item.ttl) {
```

```

        delete this.data[key];
        return null;
    }

    return item.value;
},

clear() {
    this.data = {};
}
};

```

33. Event Emitter Pattern

```

const EventEmitter = {
    events: {},

    on(event, callback) {
        if (!this.events[event]) {
            this.events[event] = [];
        }
        this.events[event].push(callback);
    },

    emit(event, ...args) {
        if (this.events[event]) {
            this.events[event].forEach(callback => callback(...args));
        }
    },

    off(event, callback) {
        if (this.events[event]) {
            this.events[event] = this.events[event].filter(cb => cb !== callback);
        }
    }
};

```

Summary and Best Practices

Key Concepts Covered

1. **Object Fundamentals:** Creation, access, modification, and deletion
2. **Object Methods:** Built-in utilities like `Object.keys()`, `Object.values()`
3. **Advanced Techniques:** Destructuring, cloning, merging, and validation
4. **Real-World Applications:** Banking, inventory, contact management systems
5. **Design Patterns:** Configuration management, caching, event handling
6. **Utility Functions:** Deep comparison, flattening, filtering, and mapping

Skills Developed

- **Object-Oriented Programming:** Understanding encapsulation and method organization
- **Data Structure Manipulation:** Working with complex nested objects
- **Algorithm Implementation:** Building efficient object processing functions
- **Design Pattern Recognition:** Implementing common programming patterns
- **Problem-Solving:** Breaking down complex requirements into object solutions
- **Code Organization:** Structuring code for maintainability and reusability

Next Steps

1. Practice each example thoroughly
2. Modify the code to explore different scenarios
3. Combine multiple patterns to build larger applications
4. Focus on understanding the underlying concepts, not just memorizing syntax
5. Build your own variations and extensions of these examples

This guide provides a comprehensive foundation for mastering JavaScript objects. Continue practicing and building upon these concepts to become proficient in object-oriented JavaScript programming.